

Building an ODI Demo with Claude Code — A Beginner's Guide

Three 1-hour sessions. Written for a first-time Claude Code user. You type plain English; Claude does the work.

What you're building

A working data demo. Over three short sessions you'll pull data out of a cloud app (like Salesforce), land it automatically in a data lake on Amazon's cloud, tidy it up, and show it in a small web app. You won't write any code yourself — you describe what you want in plain English and Claude does the work.

How Claude Code works — read this first (you're new to it, that's fine)

- 1. It's a chat in a terminal window.** You type a request in plain English and press Enter. There is nothing to memorize and no special syntax — full sentences are perfect.
- 2. It does real work for you.** It writes code, runs commands, and sets up cloud services on its own. You watch it happen and approve along the way.
- 3. It asks permission before doing anything that matters.** When it wants to run a command, a box pops up asking you to allow it. Read the one-line summary and choose **Yes / Allow**. If you're unsure, ask the person next to you before allowing.
- 4. Let it finish.** It works in steps and will often pause, show progress, or ask you a question. Wait until it stops and hands the turn back to you before typing the next thing.
- 5. You can always just talk to it.** If you don't understand something on screen, type *"explain that to me like I'm new to this"* or *"what did that just do?"* It will answer in plain language. This is the best habit to build today.

Your job vs. the driver's job

You (hands on keyboard): read each prompt below, type or paste it, press Enter, and choose **Allow** when asked. Ask questions freely.

The driver next to you: handles logins and passwords, anything that turns up as a red error, and any decision about cost or accounts. If anything looks confusing or alarming, hand it to them — nothing you type can cause real damage.

If you get stuck (keep this handy)

- A box asks to run a command and you don't know what it is → it's normal; choose Allow (or ask the driver).
- Claude asks a question you don't understand → type: *"I'm new to this — explain what you need in simple terms."*
- You see red error text → don't worry, hand it to the driver, or type: *"that looks like an error — can you fix it?"*
- Claude seems frozen → it's probably still working; give it a minute before typing again.
- You lost track of where you are → type: *"summarize what we've done so far and what's next."*

Plain-English glossary (skim it, refer back as needed)

Prompt	What you type to Claude — a plain-English instruction.
Connector	An automatic pipe that pulls data out of an app (e.g. Salesforce) for you.
OAuth	The "Sign in with Google/Salesforce" button. You log in; no passwords are shared.
Sync	The connector copying the data over for the first time.
Data lake / MDLS	Cloud storage where all the data lands, in an open format anyone can read.
Iceberg	The modern table format the data is stored as inside the lake.
AWS	Amazon's cloud. S3 stores files, Glue lists the tables, Athena runs queries, IAM is permissions.
Terraform	A tool that sets up those cloud pieces automatically from a script Claude writes.
dbt	A tool that cleans and reshapes raw data into tidy tables for analysis.
Connect Card	A Fivetran link you open in your browser to log into the source app with OAuth.

Session 1 opens with a 5-minute access check (done by the driver)

- This must happen first, or the cloud build can't start.** The programs are installed; the logins are not.
- **Fivetran:** log in, generate an API key, then in the terminal `export FIVETRAN_API_KEY=key:secret`
 - **AWS:** run `aws configure` (paste an admin access key + secret, or use SSO), then confirm with `aws sts get-caller-identity`
 - **If a login isn't available:** don't burn the hour on it. Instead, open one of the finished demos in `~/Documents/GitHub/*-ODI-Demo` and have Claude give you a guided tour — you'll still see exactly what we're building — then provision for real next session.

Carry into each session (no homework between sessions)

- The **Fivetran API key** — re-paste the export line at the start of each session (it doesn't carry over to a new terminal window).
- The driver re-runs the AWS login if needed.
- If you forget where things are, just ask Claude: *"list the demo we built and where it lives."*

Session 1 (today) — set up the cloud and pull in the data

Goal: stand up the AWS data lake, connect the source app, and start the first data sync. Before you provision, decide the blanks below.

Fill these in first: [VERTICAL] = the industry story (e.g. Banking). [ORG] = a made-up company name for the demo. [PERSONA] = who the demo is for (e.g. a risk analyst). [source] = the app to pull from (e.g. Salesforce).

Prompt 1 — kick off the build

Let's build a new ODI demo with the odi-demo-builder skill. Vertical: [VERTICAL]. Org name: [ORG]. Buyer persona: [PERSONA]. Nothing is set up yet and my AWS and Fivetran logins are now configured. Walk me through it step by step, and explain what each step does in plain language as you go.

What it does: Starts the guided build. Claude lays out the plan and begins writing the cloud setup. Expect it to ask you to allow a few commands — choose Allow.

Prompt 2 — build the cloud storage

Scaffold the Terraform for the S3 bucket, the Glue databases, and the Fivetran permissions role, then walk me through applying it. Tell me when you need anything from me.

What it does: Claude writes a setup script and runs it to create the storage and permissions on AWS. The driver may need to paste two values from the Fivetran screen when asked. You'll see a list of created resources at the end.

Prompt 3 — create the data lake destination

Create the Managed Data Lake destination in Fivetran using what Terraform just set up, and show me that it connected successfully.

What it does: Tells Fivetran where to put the data (our new lake). You'll see a confirmation with a status of connected.

Prompt 4 — connect the source app

Create the [source] connector, then give me a Connect Card link so I can sign in to [source] with OAuth.

What it does: Claude sets up the pipe from the source app and hands you a link. Open it in your browser, sign in the normal way, pick the data, and click Save. No passwords are typed into Claude.

Prompt 5 — start the data flowing

Test the connection, then start the first sync and tell me when it has finished and which tables came across.

What it does: Kicks off the first copy of data into the lake. This can take a few minutes; Claude watches it and reports when it's done.

SESSION DONE — the cloud lake is live, the app is connected, and data is flowing in. If the AWS setup ran long, stopping here is a perfectly good session 1 — we model the data next time.

Session 2 — shape the raw data into clean tables

At the start: re-paste the Fivetran key line, the driver re-checks the AWS login, and confirm last session's data arrived.

Prompt 1 — pick up where we left off

Resume the ODI demo we started. Confirm the data sync from last session finished and show me the tables that landed in the lake.

What it does: Re-orientes Claude and proves the data from session 1 is really there before we build on it.

Prompt 2 — build the data pipeline

Build the dbt pipeline to the portfolio standard — a raw layer, a cleaned layer, and a final reporting layer of tables. Apply Niraj's content rules, and explain each layer in plain language.

What it does: Claude generates the data-cleaning project that turns raw rows into tidy, business-ready tables.

Prompt 3 — run it and check the result

Run the pipeline and confirm the final tables were created. If anything fails, explain the error simply and fix it. Then show me a sample of the finished data.

What it does: Executes the pipeline and shows you the polished tables. You'll see a short sample of real data at the end.

SESSION DONE — raw data has been turned into clean, query-ready tables in the lake. Strong stopping point.

Session 3 — build the web app and tour the demo

Same quick restart at the top: re-paste the key, re-check the AWS login.

Prompt 1 — generate the web app

Generate the React frontend for the demo — a landing page with the [ORG] name and a simple diagram of the data flow, and a page that lists the connector with a link to it in Fivetran.

What it does: Claude builds a small website that presents the demo. You'll get pages you can open in a browser.

Prompt 2 — make it look sharp

Run the frontend-design pass for the [VERTICAL] industry and [PERSONA] audience — a distinctive, professional look, clean charts, and no generic AI styling.

What it does: Polishes the look and feel so it's presentation-ready rather than plain.

Prompt 3 — final quality check

Do a content quality pass: make sure the [ORG] name appears everywhere it should, no raw technical names are showing, and the connector link works.

What it does: A final cleanup so nothing looks unfinished in front of an audience.

Prompt 4 — walk the demo

Start the web app and give me a guided tour from start to finish — the landing page, the data flow, and the data in the lake.

What it does: Opens the finished demo and narrates it end to end. This is the version you'd show a customer.

SESSION DONE — a demo-ready ODI app: a live connector, a cloud data lake, a clean data pipeline, and a polished web app you can present.